

Known-Future Covariates Are the Signal: A Patch Transformer with Covariate Query Tokens for Multivariate Operational Load Forecasting

Group ID: 140

Rosela Berberi, Anja Korovesi

Deep Learning (SS26) bonus project, TU Darmstadt

Abstract

We forecast a 336-hour horizon for 96 correlated operational load series from a 168-hour history and 22 covariates. Our central observation is that on this benchmark the covariates are *known in advance* for the whole forecast window, which shifts the problem from extrapolating the target towards learning a covariate-to-load response specific to each unit. We therefore extend the PatchTST backbone with *future covariate query tokens*, a per-series embedding and a pointwise covariate residual path, and contrast it with an LSTM+Attention encoder-decoder that decodes iteratively. On held-out tails of the public training data our submitted model reaches 12.91 % WAPE in the validation regime — 65% below the seasonal-mean baseline — and 12.40 % in a harder regime reproducing the private split’s 336-step unobserved gap, cutting the naive last-value error by 75%. Two of our conclusions had to be reversed by experiments, and we report both: the recurrent decoder’s apparent collapse across the gap (18.42 % $\rightarrow$ 24.09 %) turned out to be reversible instance normalisation compounding through the rollout rather than iterative decoding; and our architecture ranking was an artefact of the compute budget, inverting once training was no longer starved. Ablations attribute almost all of the remaining gain to the covariate pathway rather than to the backbone, and a transfer to Jena Climate, where every channel except the calendar is past-only, shows how much of that margin the covariate contract rather than the architecture was buying.

1 Introduction

Multivariate time series forecasting asks for the joint future of several correlated signals given their shared past. This project’s benchmark is an operational instance: 96 anonymised units report an hourly *operational load index* and the task is to predict 336 hours ahead for each. If the index tracks pressure on a unit, as its description suggests, such a forecast is what drives staffing decisions; methodologically it is interesting because the horizon is $2\times$ the conditioning window, so no part of the answer can be read off the recent past by persistence.

Our starting point was an empirical look at the released tables rather than at the model zoo, and it changed the project. The file `validation_input.csv` covers *exactly* the rows of `forecast_index_validation.csv`: all 22 covariates are supplied for the forecast window itself, while the target is withheld. The benchmark is thus not a pure extrapolation task but a *covariate-conditioned* one, and the strongest covariate correlates 0.68 with the target at the same hour, whereas the target’s own autocorrelation has already decayed to 0.26 after six hours. A second measurement shaped the architecture: a per-series OLS fit on the covariates reaches 17.6 % WAPE against 22.1 % for a single pooled fit, i.e. the covariate-to-load response is unit-specific even though the static columns barely explain anything.

Two consequences follow: a model should consume the future covariates *directly* at the resolution at which they act, and it needs a per-unit degree of freedom. We build both into a PatchTST [17] backbone and com-

pare against the recurrent-attention hybrid from our exposé. Our contributions are **PatchTST-cov**, a patch Transformer whose forecast window enters as patched *future covariate query tokens*, with a per-series embedding and a pointwise covariate residual path (Sec. 3); a **gap-aware protocol** that scores every model both immediately after the history and across the 336-step gap separating it from the private test window (Sec. 4.1); an **ablation study** plus a permutation analysis of which covariates the model uses (Sec. 4.3); and a **transfer experiment** on Jena Climate, chosen because it inverts the covariate contract (Sec. 5).

2 Related Work

Recurrent forecasters. LSTMs [9] underpin probabilistic forecasters such as DeepAR [20], which also popularised the per-series embedding we adopt, and additive attention [1] lets a decoder re-read all encoder states. Iterative decoders suffer from *exposure bias*, for which scheduled sampling [3] is the standard mitigation and the one we use.

Transformers for long-horizon forecasting. After the original Transformer [22], a line of work attacked its quadratic cost and point-wise tokenisation of time series: Informer’s sparse attention [25], Autoformer’s decomposition and auto-correlation [23], FEDformer’s frequency-domain attention [26]. PatchTST [17] instead tokenises *sub-series patches* and processes each channel independently with a shared backbone, preserving local semantics while shortening the sequence by the patch stride;

iTransformer [13] inverts the axes and attends over variates. A sceptical counter-line matters for calibration: DLinear [24] showed a single linear layer can match several of these Transformers, and N-BEATS [18], N-HiTS [5] and TiDE [6] reach comparable accuracy with MLPs. We take that seriously: our ablations include a covariate-free variant, so the gain we claim is attributable to the covariate machinery rather than to attention per se.

Covariate-conditioned models. The Temporal Fusion Transformer [12] is the closest relative of our design: it separates static, past-only and known-future inputs and feeds the latter to the decoder; TiDE [6] does the same with a dense encoder. We borrow that taxonomy and contribute a minimal way to graft it onto a patch backbone, as query tokens on the same token sequence.

Normalisation, regularisation, optimisation. Reversible instance normalisation [11] is the near-default remedy for distribution shift between training and forecast windows; Sec. 4.3 finds it harmful at this horizon, which is one of our main results. We also use spatial dropout [21], AdamW [15] and cosine annealing with warm restarts [14], in PyTorch [19]; dilated convolutions [2] and deep state-space models [8, 7] are the natural next backbones.

3 Method

3.1 Problem setup and features

For series i and origin t we observe the target history $y_{i,t-L+1:t} \in \mathbb{R}^L$ with $L = 168$, encoder covariates $X_{i,t-L+1:t}^h \in \mathbb{R}^{L \times F_h}$, the known-future covariates $X_{i,t+1:t+H}^f \in \mathbb{R}^{H \times F_f}$ and static features $s_i \in \mathbb{R}^S$, and predict $\hat{y}_{i,t+1:t+H}$.

The 22 released covariates split into six calendar encodings (hour and day-of-week sine/cosine, a weekend flag, a normalised trend), 13 exogenous hourly drivers (demand, staffing, queue and network pressure, event load, maintenance, several risk indices) and three static columns. Ten drivers are missing at random for $\approx 4.5\%$ of rows; we interpolate within each series and append a binary *missingness indicator* per affected column, so the network can tell an imputed value from an observed one. This yields $F_h = F_f = 29$ and $S = 3$. Covariates are standardised with statistics fitted on the training slice only and serialised into the checkpoint, so inference reproduces training preprocessing without the training CSV.

The primary metric is $\text{WAPE} = 100 \cdot \sum |y - \hat{y}| / \sum |y|$, whose denominator is constant for a fixed evaluation set. WAPE is therefore a rescaled sum of absolute errors, and the aligned objective is plain L_1 in the original target units [10] — in particular not on per-window normalised targets, which would silently reweight series by their own scale.

3.2 Shared components

RevIN. Each window is normalised by its own mean and standard deviation over time and the prediction is de-normalised with the same statistics [11]. We implement it *statelessly*, passing the statistics explicitly in both directions, so that the conditioning-window statistics survive an autoregressive rollout: caching them on the module — as reference implementations do — lets the decoder’s own call on a single timestep overwrite them with $(\mu, \sigma) = (y_t, 0)$.

Series embedding. A learned vector $e_i \in \mathbb{R}^{16}$ per series is concatenated to s_i [20]. Row 0 is reserved for unseen series, so a checkpoint degrades gracefully rather than failing if the private split introduces a new unit.

3.3 PatchTST-cov

The backbone follows PatchTST [17]: the RevIN-normalised target history is cut into overlapping patches of length $P = 24$ with stride 12, giving $N = 13$ tokens, each linearly embedded to $d = 128$. Channel independence is preserved in the sense of the original paper — one shared backbone sees a single univariate target channel and every series is an independent sample. Encoder covariates are fused into the history tokens by flattening the corresponding $P \times F_h$ block and projecting it to d .

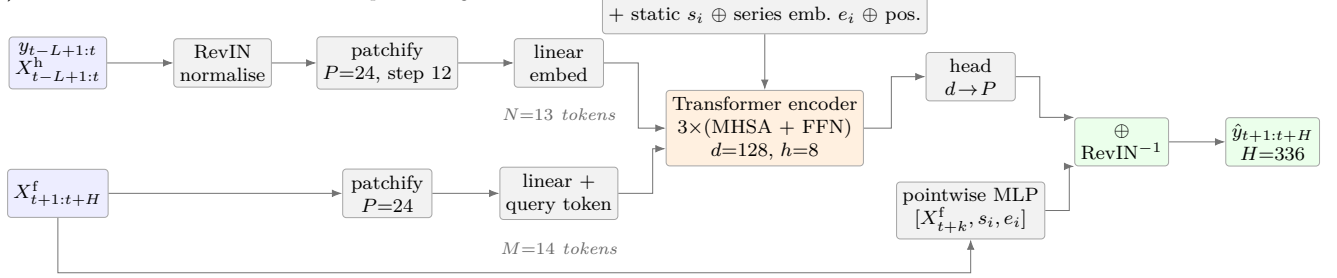
The extension is on the decoder side (Fig. 1a). We patch the known-future covariates into $M = H/P = 14$ non-overlapping blocks, embed each, add a learned query token, and *append* them to the same token sequence. Self-attention then relates every forecast segment both to the observed history and to the other forecast segments, and a shared head maps each query token’s output to its P target values. The forecast is therefore **direct**: no autoregression inside the horizon, hence no error accumulation.

A patch token compresses 24 hours of covariates into one 128-dimensional vector before the head expands it back to 24 predictions, discarding within-patch resolution. Because the drivers act largely instantaneously, we add a *pointwise residual path*: a two-layer MLP applied independently at each forecast step to $[X_{t+k}^f, s_i, e_i]$, summed onto the patch prediction in normalised space, so the Transformer models only the residual cross-step structure. This is the largest departure from our exposé’s plan.

3.4 LSTM+Attention

The recurrent branch (Fig. 1b) is the encoder–decoder from the exposé, adapted to covariates. A two-layer LSTM (128 units) encodes $[\text{RevIN}(y), X^h, s_i, e_i]$ after spatial dropout; a single-layer LSTM decoder unrolls 24 steps, at each step consuming the known covariates of the step being predicted, an additive attention context over all encoder states [1], and either its own previous prediction or the ground truth with probability ϵ . We anneal ϵ linearly from 0.9 to 0.1 across training [3]. Hori-

(a) **PatchTST-cov** — *direct multi-step decoding*



(b) **LSTM+Attention** — *iterative decoding in 24-step blocks*

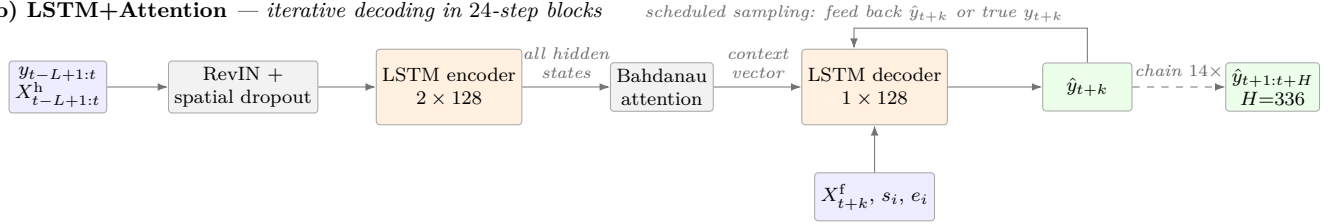


Figure 1: The two architectures. (a) **PatchTST-cov** patches the target history *and* the known-future covariates and lets one encoder attend over both; the head reads the horizon off the query tokens (a direct 336-step forecast) and a pointwise MLP on X^f restores the resolution patching discards. (b) **LSTM+Attention** decodes 24 steps at a time and chains blocks. Both share the static/series embedding and the raw-scale L_1 objective; the RevIN stage is removed in the final models (Sec. 4.3).

zons beyond one block are covered by chaining: predictions are appended to the conditioning window together with the known covariates of those steps, and the window is trimmed back to L .

3.5 Ensemble, training and the gap

The two families make structurally different errors, so we blend their rollouts before clipping, with weights from greedy forward selection with replacement [4]. Which split that selection runs on turns out to matter more than the values it returns, and Sec. 4.3 returns to it. Members are trained separately with AdamW (lr = 10^{-3} , weight decay 10^{-4}), gradient clipping at 1.0, batch size 64, cosine annealing with warm restarts [14] and L_1 loss. Training windows are cut with a stride that trades epoch cost against coverage: the shipped members use 2–6 (PatchTST-cov, up to 151k windows per epoch) and 6–12 (LSTM, up to 55k). Predictions are clipped from below at the smallest target seen in training, since the load index is strictly positive.

At private evaluation the input directory provides only the test covariates and the forecast index, and the last *released* target lies 336 steps earlier. Our checkpoint therefore bundles a 504-step tail of the public history (with targets) plus the public validation covariates; `predict.py` rebuilds one continuous per-series timeline and rolls forward to the end of the requested index — two chained 336-step blocks for PatchTST-cov, 28 for the LSTM.

4 Experiments on the benchmark

4.1 Protocol

The public release contains no validation labels, so *every* number we report comes from a chronological split of `train.csv` (96 series $\times$ 4,320 hourly steps, 2023-01-01 to 2023-06-29), whose last 672 steps are held out and split in two. The **validation regime** scores steps 3,649–3,984 with the forecast origin at the end of the fit slice, mirroring the public leaderboard; the **test regime** scores steps 3,985–4,320 from the *same* origin, so the model must first bridge 336 unobserved steps, mirroring the private split. Model selection — early stopping, blend weights, all hyperparameters — uses the validation regime only; the test regime is scored once per final checkpoint. The evaluation *set* is a held-out tail rather than the hidden validation split, so these values are not directly comparable to leaderboard scores, but in practice they track it: the protocol predicted our leaderboard WAPE to within 0.05.

Two compute budgets. We report two, because the difference between them turned out to be a first-order effect. The **constrained** budget is what a CPU allows: a coarse window stride, so each epoch sees a few per cent of the available windows. The **full** budget is a 26-configuration grid on one RTX 2080 Ti at strides 2–12, up to 151k windows per epoch. The ablations (Sec. 4.3) use the full budget too.

4.2 Main results

Table 1 collects the results. Every model clears the mandatory naive last-value reference and the stronger

Table 1: Benchmark results on the held-out tails of `train.csv`; lower is better, WAPE is the leaderboard’s primary metric, and the test regime starts 336 steps after the conditioning window. Each architecture appears twice, at the two training budgets of Sec. 4.1; the parenthesised count is training windows per epoch. * marks the variants that entered the shipped blend; full six-metric tables are in `results/`.

Method	Validation		Test (gap)	
	MAE	WAPE	MAE	WAPE
Naive last value	4.91	52.34	5.34	49.96
Seasonal naive, lag 24	5.37	57.18	4.94	46.26
Seasonal naive, lag 168	5.15	54.88	4.56	42.74
Seasonal mean, dow $\times$ hour	3.50	37.27	3.35	31.38
LSTM+Attention (7k win.)	1.73	18.42	2.57	24.09
LSTM+Attention (no RevIN, 55k win.)*	1.24	13.16	1.34	12.58
PatchTST-cov (13k win.)	1.49	15.92	1.66	15.56
PatchTST-cov (no RevIN, 151k win.)*	1.31	13.93	1.44	13.52
Blend of 5 runs (submitted)	1.21	12.91	1.32	12.40

seasonal-mean baseline by a wide margin; our submitted blend of 5 runs reaches 12.91 % WAPE in the validation regime against 37.27 % for the seasonal mean and 52.34 % for the naive baseline, and 12.40 % in the gap regime — a 75 % error reduction over the reference we are required to beat. The paired rows are the interesting part: each pair is one architecture under two budgets, and they carry the two results that forced us to revise our conclusions.

The compute budget, not the architecture, decided our first comparison. Our exposé asked whether direct multi-step output beats iterative decoding. Under the constrained budget — 7k windows per epoch for the LSTM, 13k for the Transformer — the answer looked decisive: the Transformer reached 15.92 % while the recurrent model managed only 18.42 % and collapsed to 24.09 % across the gap. Re-run at the full budget, with 8 $\times$ and 12 $\times$ the windows, the ranking *inverts*: the LSTM reaches 13.16 % and the Transformer 13.93 %. Neither number was a property of the architecture; both were properties of how starved it was. The failure modes were in fact opposite, which is why no single budget separates them fairly: the recurrent model was under-trained (its best epoch was always its last), while the Transformer was already overfitting (best epoch 3–6 of 40–60). The fix for one is the opposite of the fix for the other — more data per epoch for the LSTM, more dropout for the Transformer, worth 14.55 $\rightarrow$ 13.93 %. We report it because “patch Transformers beat recurrent models here” is a conclusion our own first experiment would have supported and our second refutes.

The gap-regime collapse was a normalisation bug, not error accumulation. The same starved comparison also appeared to confirm that iterative decoding accumulates error: the LSTM lost 5.7 WAPE chaining 28 blocks instead of 14. Sec. 4.3 traces almost all of it to RevIN, and Fig. 2 shows the mechanism: fitting a line to MAE against the forecast step, the pre-repair model

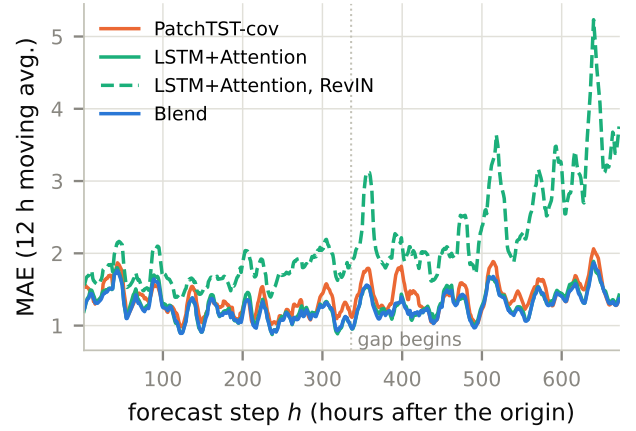


Figure 2: MAE against the forecast step h (12-hour moving average). The two held-out windows share one origin and are contiguous, so $h = 1 \dots 336$ is the validation regime and $h = 337 \dots 672$ the test regime. Only the pre-repair recurrent model (dashed) drifts upward with h . Fitted drift per 100 steps: +0.258 MAE with RevIN versus +0.020 (LSTM), +0.025 (PatchTST-cov) and +0.022 (blend) without it.

drifts upward by +0.26 MAE per 100 steps while the repaired LSTM, the Transformer and their blend drift by +0.02 — an order of magnitude apart, and flat to within the noise of the curve.

Blending pays because the failure modes differ. With both families repaired and properly trained they land within 0.8 WAPE of each other while making differently-shaped errors, so the greedy selection of Sec. 4.3 over five runs beats its own best member by 0.25 WAPE.

4.3 Ablations and analysis

The covariate contract dominates everything else. Removing the future covariate tokens costs 18.7 WAPE points and removing the covariates entirely 18.8; the recurrent model loses 20.6 the same way, 26.6 across the gap. No other edit in Table 2 moves either model by more than 4.2, and most by less than 1. This is the empirical core of the paper: the backbone is not what produces the accuracy — the availability of future covariates is — and the architecture’s job is to consume them at the right resolution.

RevIN hurts both families. Adding it back costs the Transformer +0.32/+0.77 and the recurrent model +2.83/+4.80; the asymmetry is the clue. RevIN re-injects the conditioning window’s own mean and standard deviation into the prediction, and over 336–672 steps last week’s level is a poor estimate of next fortnight’s, so the transform adds variance rather than removing shift. For the *iterative* decoder it is worse: every chained block re-estimates those statistics *from the block it just predicted*, so a level error re-enters the normaliser and is amplified at each of the 28 hops — the mechanism

Table 2: Ablations at the full budget. Both references are the shipped configuration — RevIN-free, stride 6, 40/30 epochs — and reproduce their Table 1 rows exactly, so every Δ answers “what if I change this one thing about the submitted model”. RevIN is therefore *added* rather than removed. Positive Δ is worse. “No future cov. tokens” also removes the pointwise path.

Configuration	Validation		Test (336 h gap)	
	WAPE	Δ	WAPE	Δ
PatchTST-cov (ref.)	14.76		13.68	
+ RevIN	15.08	+0.32	14.45	+0.77
MSE instead of L_1	15.23	+0.47	14.07	+0.39
no covariates (vanilla)	33.55	+18.80	32.65	+18.97
no future cov. tokens	33.42	+18.66	30.36	+16.68
no pointwise path	15.36	+0.61	14.38	+0.70
no series embedding	15.22	+0.46	14.37	+0.69
patch 48, step 24	14.55	-0.21	13.87	+0.19
patch 8, step 4	14.31	-0.45	13.35	-0.33
LSTM+Attention (ref.)	13.16		12.58	
+ RevIN	15.99	+2.83	17.38	+4.80
free running only	13.53	+0.37	12.81	+0.24
no attention	13.30	+0.14	12.61	+0.03
no covariates	33.73	+20.57	39.14	+26.57
no series embedding	17.27	+4.11	24.33	+11.75
teacher forcing only	14.14	+0.98	13.51	+0.94

behind the collapse we first attributed to iterative decoding. The sign is the same on validation, so we could act on it without consulting the test split; no member of the shipped blend uses RevIN.

Two components swapped places when we stopped starving the models. We first ran this table at a $16\times$ coarser stride, and two rows read very differently there. *Attention* looked load-bearing for the recurrent branch (+11.03 without it); at the full budget it is worth +0.14/+0.03 — nothing. *The per-series embedding* looked worthless (−0.12, within noise); at the full budget it is the most valuable thing the LSTM has after the covariates, +4.11/+11.75. Both reversals point the same way as the architecture comparison in Sec. 4: a starved model cannot exploit capacity, so components that supply capacity look useless and their substitutes look essential. The starved numbers were not merely noisier, they were misleading in a systematic direction. The embedding result also settles a question from our exposé — the per-unit response the OLS diagnostic predicted is real, but the Transformer absorbs it through the covariate pathway (+0.46/+0.69) while the recurrent decoder, seeing one step at a time, needs it stated explicitly.

Scheduled sampling helps, and couples the schedule to the budget. Both endpoints of the anneal are worse than the anneal itself: teacher forcing only ($\epsilon = 1$) costs +0.98/+0.94 — the classic exposure-bias failure — and free running only ($\epsilon = 0$) costs +0.37/+0.24. The schedule has a side effect we did not anticipate, though: ϵ is annealed across the *whole* epoch budget, so a longer budget is not a superset of a shorter one but a slower anneal. Stretching the winning 30-epoch LSTM to 100 epochs made it *worse* (13.16 $\rightarrow$ 13.40), and the usual

guard — more epochs cannot hurt, early stopping will catch it — fails when the objective itself moves with the epoch index. Finally, replacing L_1 with MSE costs +0.47/+0.39 under WAPE, as the metric-alignment argument of Sec. 3 predicts.

Where the design is not optimal. Halving the patch to $P = 8$ improves both regimes (−0.45/−0.33), so the length we inherited from PatchTST is not the right one here — unsurprising, since finer patches do the same job as the pointwise residual path (+0.61/+0.70), which exists precisely because $P = 24$ discards within-patch resolution. We report it rather than quietly adopting it: it arrived after the blend was built, and acting on it now would be tuning on the split we report.

Blend weights are chosen on a gap-matched criterion. Which split the greedy selection runs on matters more than the weights it returns: a weight tuned on the validation regime is tuned on a regime the private split never runs in. We therefore select on a *gap-matched* criterion — the same labels, but with the origin moved back one window so members must bridge 336 unobserved steps first (12.93% for the blend, against 12.91% ungapped). With RevIN-free members the two criteria agree; before the repair they did not, because RevIN’s damage is invisible without a gap. The weights land on 0.36/0.24/0.12 over three LSTM runs and 0.24/0.04 over two PatchTST runs — and those two Transformers are individually the *weakest* in the pool (13.93 and 14.55 against 13.16), so the blend buys decorrelated errors, not accuracy.

What the model uses. Shuffling one covariate at a time across the forecast window gives a concentrated ranking: from a base of 13.93%, `queue_pressure_forecast` alone is worth +15.9 WAPE and `network_pressure_forecast` +8.7, then `maintenance_known` (+4.5), `demand_forecast` (+3.0) and `promotion_intensity` (+2.3); nothing else reaches +2.0 and the calendar encodings are negligible ($\leq +0.26$). Two of the 19 covariates carry most of the accuracy. The values are a per-channel lower bound, since the covariates are correlated; the full ranking is in [figures/permutation_importance.pdf](#).

5 Additional dataset: Jena Climate

Dataset and use case. The Jena Climate dataset [16] records 14 atmospheric variables — pressure, air and potential temperature, dew point, humidity, three vapour-pressure channels, air density, wind speed and direction — every 10 minutes at the Max Planck Institute for Biogeochemistry weather station, 2009-01-01 to 2017-01-01 (420,551 rows). The use case is local weather nowcasting: the next two weeks of temperature, and of pressure as a second target, from the recent multivariate state.

We chose it because it *inverts* the benchmark’s covari-

Table 3: Jena Climate, hourly, $L = 168$, $H = 336$, rolling-origin evaluation. Temperature crosses zero, so MAE replaces WAPE; RMSE is in `results/jena.json`. Both targets use the same architecture and optimiser settings as the benchmark, with a reduced epoch budget and no static or missingness features; the calendar encodings are the only known-future covariates.

Method (MAE)	temperature [°C]		pressure [mbar]	
	val	test	val	test
Naive last value	3.79	5.96	8.04	7.88
Seasonal naive, lag 24	3.53	5.00	8.59	8.09
Seasonal mean	5.56	6.69	6.73	6.49
PatchTST-cov (ours)	3.04	3.76	7.23	6.42
LSTM+Attention (ours)	2.94	3.79	8.19	7.00

ate contract — nothing except the calendar is known in advance — so it tests whether our gains come from modelling or from unusually generous inputs. It is also harder as a multivariate problem: the channels are non-linearly coupled (dew point and vapour pressure are functions of temperature and humidity) and the target has diurnal *and* annual seasonality, the latter longer than any window we condition on.

What had to change. Exactly one abstraction: a *past-only* covariate group. Encoder features become $[\text{known-future} \cup \text{past-only}]$ while decoder features are the known-future subset alone, so $F_h = 24 > F_f = 8$; both architectures read the two widths from the specification and no layer was rewritten. When the iterative decoder appends a predicted block to its window it cannot know the past-only channels there, so they are filled by persistence — never by their true values, which would leak the answer. Preprocessing is dataset-specific: hourly means, repair of the -9999 wind sentinel, Cartesian wind components, day-of-year sine/cosine. The split is chronological 70/10/20, scored rolling-origin (one origin every 336 steps); temperature crosses zero, so we report MAE rather than WAPE.

Does it transfer? Only partly, and where it fails is the useful result (Table 3). On *temperature* it transfers cleanly: both models beat every baseline on both splits, cutting test MAE from 5.00 to 3.76, a 25% reduction, with no tuning beyond the feature specification. On *pressure* it essentially fails: PatchTST-cov is worse than the seasonal mean on validation (7.23 vs 6.73) and only ties it on test, and the recurrent model loses on both.

The contrast is what our thesis predicts. With only the calendar known in advance, the query tokens and the pointwise path have almost no input and the model falls back on target history — precisely the regime in which linear baselines are competitive [24]. What survives then depends on how much of the target that history explains: temperature has a diurnal cycle the encoder can exploit, whereas pressure is driven by synoptic systems invisible in a 168-hour window and whose long-run mean the seasonal baseline already captures. The two decoders are

again within 0.03 MAE on temperature, a third sign that the direct/iterative choice is not what separates models. Backbone, recipe and past-only abstraction transfer unchanged; the *size* of the win moves with the covariate contract.

6 Conclusion

We set out to compare a recurrent-attention hybrid against a patch Transformer, and the most useful results were not about which backbone wins. Because the benchmark supplies all 22 covariates for the forecast window, the task is covariate-conditioned regression rather than extrapolation, and consuming that window at the right resolution accounts for essentially the whole improvement: every other component in Table 2 is worth an order of magnitude less, attention itself worth $+0.14$.

Our other results are corrections to conclusions we had already drawn. The gap-regime collapse we would have reported as evidence for direct decoding was RevIN compounding through the rollout. And three rankings — the two architectures, attention against none, the per-series embedding against none — came out differently once the models were no longer starved, always in the same direction: starvation makes capacity look useless and its substitutes essential. A blend of 5 runs reaches 12.40% WAPE across the gap and 12.82% on the public leaderboard, within 0.09 of what our local protocol predicted — which is the real result here: the protocol, not any one number.

Limitations. Selection rests on a single 336-step validation window with few seed repeats; re-scoring identical checkpoints under a different thread count moves WAPE by up to 0.07, so differences below ≈ 0.1 are not real and the top three blend members sit inside that band. Every ablation is a single run, and having watched two of them reverse with the budget we would not defend any Δ under ≈ 0.5 without seeds. The gap handling assumes the private window is contiguous with the public history. Finally the model is deterministic and trained on L_1 , hence predicts a conditional median: on this right-skewed target it under-predicts the total by $\approx 7\%$ by construction, optimal for WAPE and wrong for anything volume-based. A quantile head [20] is the principled fix; a finer patch (Sec. 4.3), cross-series attention over the 96 units and an MLP covariate baseline such as TiDE [6] are what we would try next.

Contributions

Rosela Berberi — covariate-contract analysis; feature pipeline; PatchTST-cov; training and evaluation harness including the gap-aware protocol; inference and submission packaging. **Anja Korovesi** — LSTM+Attention including the RevIN statefulness fix, scheduled sampling and the chained rollout; baselines and metrics; ablation and permutation studies; Jena Climate; figures. Experiments were designed and the report written jointly.

References

- [1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*, 2015.
- [2] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- [3] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems*, 2015.
- [4] Rich Caruana, Alexandru Niculescu-Mizil, Geoff Crew, and Alex Ksikes. Ensemble selection from libraries of models. In *International Conference on Machine Learning (ICML)*, 2004.
- [5] Cristian Challu, Kin G. Olivares, Boris N. Oreshkin, Federico Garza, Max Mergenthaler-Canseco, and Artur Dubrawski. NHITS: Neural hierarchical interpolation for time series forecasting. In *AAAI Conference on Artificial Intelligence*, 2023.
- [6] Abhimanyu Das, Weihao Kong, Andrew Leach, Shaan Mathur, Rajat Sen, and Rose Yu. Long-term forecasting with TiDE: Time-series dense encoder. *Transactions on Machine Learning Research*, 2023.
- [7] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modeling*, 2024.
- [8] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.
- [9] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [10] Rob J. Hyndman and Anne B. Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.
- [11] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International Conference on Learning Representations*, 2022.
- [12] Bryan Lim, Sercan Ö. Arik, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021.
- [13] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. iTransformer: Inverted transformers are effective for time series forecasting. In *International Conference on Learning Representations*, 2024.
- [14] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017.
- [15] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [16] Max Planck Institute for Biogeochemistry. Jena climate dataset: Weather station data, 2009–2016. <https://www.bgc-jena.mpg.de/wetter/>, 2016. Accessed August 2026.
- [17] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. In *International Conference on Learning Representations*, 2023.
- [18] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations*, 2020.
- [19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 2019.
- [20] David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3):1181–1191, 2020.
- [21] Jonathan Tompson, Ross Goroshin, Arjun Jain, Yann LeCun, and Christoph Bregler. Efficient object localization using convolutional networks. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2015.
- [22] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [23] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems*, 2021.
- [24] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *AAAI Conference on Artificial Intelligence*, 2023.
- [25] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI Conference on Artificial Intelligence*, 2021.
- [26] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International Conference on Machine Learning*, 2022.